
Final Project Report

Jay Dadhiala

1010147431, jay.dadhiala@mail.utoronto.ca

Github: <https://github.com/JaySinghD/AI-URL-Security-Checker>

1. Introduction

With the increase of cybercrime since the Covid-19 pandemic (1), it is more important than ever to be safe when using the internet. Cybercriminals use phishing links and (more generally) malicious links to steal personal information, which can be sold on the deep web, used to steal money from your bank account, or download malware onto your system. To help reduce the effectiveness of malicious links, I am developing a deep learning model which will notify an internet user whether a link is safe or malicious. Deep learning is a reasonable approach for this task because malicious URLs contain complex patterns that are difficult to capture with hand-crafted features. Neural networks have shown high level performance in similar security classification tasks like spam filtering and phishing detection. For instance, convolutional neural networks (CNNs) applied to phishing website classification achieved 98.2% detection accuracy, which surpasses traditional classifiers on the same dataset (2).

2. Illustration

The following figure (Figure 1) illustrates the core idea behind my model and its architecture.

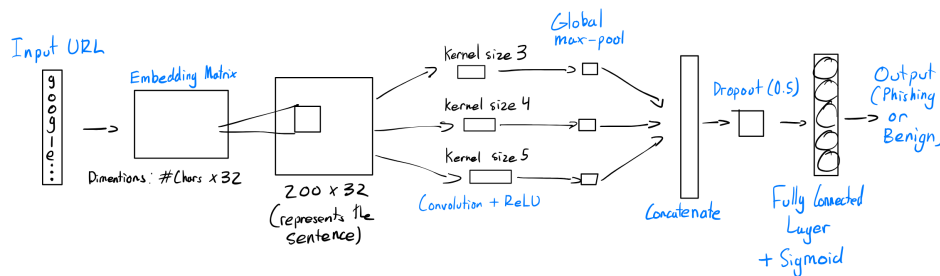


Figure 1: Overview of the Model

3. Background & Related Work

The following section gives brief descriptions of 5 related works in the field.

- 1. Yerima & Alzaylaee (2):** Uses CNNs for phishing website detection, showing 98.2% detection rate and improvement over traditional ML classifiers.
- 2. Ma et al. (3):** Uses ML to detect malicious sites via URL text and host info (domain age, IP reputation), showing better generalization than blacklists.
- 3. Sahoo, Liu, Hoi (4):** A comprehensive survey arguing that machine learning directly on URLs is superior to insufficient static blacklists.
- 4. Le et al. (5):** Proposes URLNet (CNN-based) to detect character-level tricks (weird symbols) and word-level meanings, improving over older methods.
- 5. Maneriker et al. (6):** Uses Transformers (URLTran) for phishing detection, achieving a TPR of 86.6% vs 71.2% baseline at very low false positive rates.

4. Data Processing

For training and validation data, I used the URL-Phish dataset (7). URL-Phish contains 100,000 benign URLs collected from trusted domains, and 11,660 phishing URLs obtained primarily from

PhishTank. The dataset has already been preprocessed (duplicate removal, invalid entry exclusion, normalization).

To process this dataset so it better aligns with my needs, I removed all URL attributes except for whether a URL is benign (0) or phishing (1). I did this because the manually extracted attributes were tabular data, whereas my CNN architecture is specifically designed to extract its own features from sequential data (the raw character string). Furthermore, I removed a large amount of benign URLs to ensure that the number of phishing URLs was the same as the number of benign URLs. From there, I created two balanced datasets (50% benign, 50% phishing): the training set (80%) and the validation set (10%). Samples from the training set can be seen in Table 1. The testing set (10% of total) was obtained from a similar dataset, PhiUSIIL (8), detailed in Section 9.

Table 1: Example cleaned URL Samples from Training Dataset. Asterisks for safety.

URL	Label
http://huygens.supercomputer.nl/NCF/	0
http://www.mologen.com/	0
https://l.*****.co/l?u=https://qr-codes.io/tZyfa5	1
https://pv.*****.sa.com/js/webmail.aruba.it/...	1

5. Architecture

The primary model is a character-level Convolutional Neural Network, designed to capture local sequential patterns within URLs (such as ".exe", "login", or obfuscated IP addresses). The architecture consists of four main stages:

Input Processing and Embedding: URLs are tokenized at the character level. Each URL is truncated/padded to a fixed length of 200 characters. A learnable Embedding Layer transforms indices into dense vectors of size 32.

Multi-Channel Convolutional Layers: I used 1D convolutions with three parallel layers (kernel sizes 3, 4, and 5) to capture 3, 4, and 5 characters at a time. Each layer uses 128 filters.

Global Max Pooling: Applied to each channel to extract the strongest signal, discarding positional information.

Classification Head: Pooled outputs are concatenated (size $128 \times 3 = 384$). A Dropout layer (0.5) prevents overfitting. A fully connected layer maps the vector to a single logit passed through a Sigmoid activation.

Note: The loss function is binary cross entropy with logits. The optimizer is Adam (LR = 0.001).

6. Baseline Model

I trained a logistic regression classifier based on the URL’s lexical features. I identified features associated with Phishing URLs: URL length, special characters (., /, @, ?), digits, and keywords (login, secure, etc.). The model extracts statistical counts based on these features.

Using the processed training and validation datasets, I obtained the following results:

Table 2: Baseline Logistic Regression Results on Validation Set

Actual	Predicted Class		Performance Metrics		
	Benign	Phishing	Prec.	Recall	F1
Benign	1117 (TN)	179 (FP)	0.77	0.86	0.82
Phishing	327 (FN)	969 (TP)	0.84	0.75	0.79
Overall	Accuracy: 80.48%		Weighted F1: 0.80		

7. Quantitative Results

To measure performance, I used Accuracy, Precision, Recall, and F1-Score. In Figure 2, we see training and validation loss decreasing over time and closely linked, indicating convergence without overfitting. Interestingly, validation loss is consistently lower than training loss, likely due to the validation set containing fewer difficult edge cases than the diverse training set.

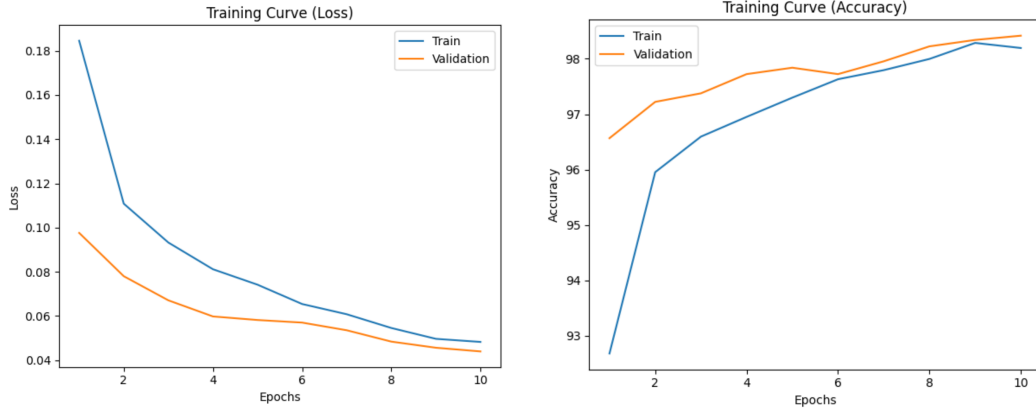


Figure 2: Loss Curve (Left) and Accuracy Curve (Right) for the CNN Model.

From Table 3, the CNN validation accuracy is 98.42%. High precision (0.99) for phishing URLs means the model is very sure when flagging phishing. Only 12 false positives occurred, meaning legitimate sites are rarely flagged.

Table 3: Primary CNN Model Results on Validation Set

Actual	Predicted		Metrics		
	Benign	Phishing	Prec.	Recall	F1
Benign	1273	23	0.99	0.98	0.98
Phishing	18	1278	0.98	0.99	0.98
Overall	Acc: 98.42%		Weighted F1: 0.98		

As shown in Table 4, the CNN provided a +17.94% accuracy increase and +0.19 F1-score improvement, demonstrating its improved ability to handle URL feature sequences.

Table 4: Model Comparison: Baseline vs. CNN

Model	Acc.	Prec. (P)	Rec. (P)	F1 (P)
Baseline	80.48%	0.84	0.75	0.79
CNN	98.42%	0.98	0.99	0.98
Improvement	+17.94%	+0.14	+0.24	+0.19

8. Qualitative Results

The confidence score is derived from the sigmoid activation, representing the probability that the URL is phishing.

Table 5: Numbered Sample Model Predictions

#	URL Sample	Act.	Pred.	Conf.	Analysis
1	sms0909v.weebly.com	Phish	Phish	100%	Success (TP). Detects free hosting subdomain.
2	irsl.u-paris.fr/...	Benign	Phish	98.9%	Failure (FP). Flagged due to length/hyphens.
3	irsverificationidme.com	Phish	Benign	36.7%	Failure (FN). Missed semantic "irs" thread.
4	nacw.gov.kh/...	Phish	Benign	9.0%	Failure (FN). Trusted .gov domain.

From Table 5, we see various predictions. For URL 1, the model observes the free hosting subdomain (weebly.com) often used by scammers. URL 2 is mistakenly flagged due to excessive length and hyphens. URL 3 fails because the model did not recognize the semantics of "irs" and "verification" without structural anomalies. URL 4 failed as the model trusted the government domain (.gov.kh).

9. Evaluation of Model on New Data

To evaluate generalization, I acquired an independent dataset from the PhiUSIIL repository (8), consisting of 2,592 URLs (50/50 split of benign and phishing), never seen during training.

Table 6: Evaluation on New Data (PhiUSIIL Dataset)

Model	Acc.	Prec. (P)	Rec. (P)	F1
Baseline	75.42%	0.94	0.54	0.69
CNN	77.93%	0.99	0.56	0.72
<i>Improvement</i>	<i>+2.51%</i>	<i>+0.05</i>	<i>+0.02</i>	<i>+0.03</i>

As shown in Table 6, the CNN (77.93%) outperformed the baseline (75.42%). Notably, the CNN achieved near-perfect Precision (0.99), having only 4 False Positives across 1,296 benign samples. This indicates high reliability. However, both models dropped in Recall (56%) compared to validation, suggesting the test set contains complex phishing structures differing from the training distribution. Despite this, the CNN maintained a higher F1-score.

10. Discussion of Results

Distribution Shift and Generalization: The discrepancy between the Validation Accuracy (98%) and the New Data Accuracy (78%) shows a clear distribution shift. The model likely overfitted to specific syntactical features present in the source dataset (URL-Phish). When exposed to the PhiUSIIL dataset, which likely contains different phishing techniques (e.g. unseen URL shorteners), the CNN model struggled to adapt, resulting in a Recall drop to 56%. This confirms that while the model learns local patterns effectively, it does not have the semantic understanding to generalize to datasets with different styles of phishing attacks.

Precision vs. Recall in Security: Despite the lower recall, the CNN’s superior Precision (0.99) is a great success. A False Positive (blocking a legitimate URL) can be very disruptive to people and to businesses. The model acts like a conservative filter, in that it rarely raises a false alarm. So this model could be used as a high-confidence layer in a larger security stack as a sort of final check, whereas a low precision model could send many bothersome and disruptive false alerts.

11. Ethical Considerations

There is a chance a user experiences a false negative (no indication the URL is malicious). This could lead to data theft or malware. It must be clear to the user that the model is not 100% accurate so they perform due diligence. Furthermore, incorrect training data could lead to biases, causing the model to fail in protecting against specific types of phishing links.

References

- [1] S. Monteith, M. Bauer, M. Alda, J. Geddes, P. C. Whybrow, and T. Glenn, “Increasing cybercrime since the pandemic: Concerns for psychiatry,” *Current Psychiatry Reports*, vol. 23, no. 1, p. 18, 2021.
- [2] S. Y. Yerima and M. K. Alzaylaee, “High accuracy phishing detection based on convolutional neural networks,” *arXiv preprint arXiv:2004.03960*, 2020.
- [3] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious urls,” in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, (Paris, France), pp. 1245–1253, ACM, 2009.
- [4] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious url detection using machine learning: A survey,” *arXiv preprint arXiv:1701.07179*, 2017.
- [5] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “Urlnet: Learning a url representation with deep learning for malicious url detection,” *arXiv preprint arXiv:1802.03162*, 2018.
- [6] P. Maneriker, J. W. Stokes, E. Garcia Lazo, D. Carutasu, F. Tajaddodianfar, and A. Gururajan, “Urltran: Improving phishing url detection using transformers,” in *MILCOM 2021 – 2021 IEEE Military Communications Conference (MILCOM)*, pp. 197–204, IEEE, 2021.
- [7] S. Y. Yerima, “Phishing url dataset,” 2020. Accessed: December 3, 2025.
- [8] M. Kowalski and J. Rumiński, “Phiusiil phishing url dataset,” 2024. Accessed: December 3, 2025.